МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3 по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Д. М. Мишин
Группа: М8О-307БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Загрузить изображения из файловой системы, создать текстуры из изображений и, используя текстурные координаты в вершинах, а также сэмплеры, наложить их на рисуемые объекты.

Вариант 18. Сфера.

Метод решения

За основу использована сцена второй лабораторной работы: процедурно построенная сфера и плоскость пола, свободная камера и освещение по модели Блинна–Фонга. В данной работе к объектам добавлено текстурное альбедо, а для сферы реализован анимированный эффект сэмплирования.

Структура вершины дополнена двумерными текстурными координатами UV и содержит позицию, нормаль и UV, всего 32 байта. Сфера строится из 32 полос и 64 секторов и содержит 2145 вершин и 12288 индексов. Координаты формируются одновременно с геометрией: номер сектора определяет координату u, номер полосы — координату v.

$$u = j / 64, \quad v = i / 32, \quad 0 \le j \le 64, \quad 0 \le i \le 32.$$

На шве сферы вершины дублируются с координатами u = 0 и u = 1. Для пола заданы четыре пары UV в диапазоне от 0 до 8, благодаря чему изображение повторяется восемь раз вдоль каждой стороны плоскости. Вершинный шейдер передаёт UV во фрагментный шейдер для интерполяции внутри треугольников.

При запуске из каталога assets/lab3 загружаются шесть PNG-изображений: Checkerboard, Brick, Wood, Marble, Tiles и Waves. Чтение и декодирование выполняются библиотекой libpng; изображения преобразуются в четырёхканальный формат RGBA. Для каждой текстуры создаются VkImage в формате VK_FORMAT_R8G8B8A8_SRGB и соответствующий VkImageView.

Пиксели сначала записываются в staging-буфер, доступный процессору. Затем изображение переводится из VK_IMAGE_LAYOUT_UNDEFINED в VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, а команда vkCmdCopyBufferToImage копирует пиксели в память GPU. После копирования изображение переводится в VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL. Барьеры задают зависимость между записью при переносе и последующим чтением

во фрагментном шейдере. Staging-буфер освобождается после завершения команд загрузки.

Все изображения используют общий VkSampler с линейной фильтрацией и режимом адресации VK_SAMPLER_ADDRESS_MODE_REPEAT. При обычном сэмплировании шейдер читает цвет texture(albedo_texture, UV), умножает его на выбранный оттенок материала и использует результат как альбедо для фонового и диффузного освещения. Цвет и коэффициент блика сохраняются параметрами материала из предыдущей работы.

Данные сцены и материалов разделены на два типа наборов дескрипторов. В наборе сцены set = 0 находятся uniform-буфер общих параметров и два storage-буфера источников света. Для каждой из шести текстур выделяется собственный набор set = 1: binding = 0 содержит дескриптор динамического uniform-буфера моделей, binding = 1 — комбинированный дескриптор изображения и сэмплера. Таким образом, оба объекта могут независимо выбирать текстуры и совместно использовать один материал.

Свойства сферы и пола размещаются в одном динамическом uniform-буфере. Каждая структура занимает 160 байт и содержит матрицу модели, матрицу нормалей, оттенок альбедо и параметры блика. Расстояние между структурами выравнивается по ограничению minUniformBufferOffsetAlignment физического устройства:

$$stride = ceil(160 / alignment) \cdot alignment.$$

При вызове vkCmdBindDescriptorSets выбирается набор дескрипторов текущей текстуры и передаётся динамическое смещение: 0 для сферы и stride для пола. Размер диапазона дескриптора равен размеру одной структуры модели. Поэтому при смене текстуры достаточно привязать другой набор, а отдельный uniform-буфер для каждого объекта не требуется.

Вершинные и индексные данные также перенесены в память с флагом VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT. Для их загрузки создаются staging-буферы, выполняется vkCmdCopyBuffer и устанавливаются барьеры от записи переноса к чтению вершинных атрибутов или индексов. После завершения копирования временные буферы освобождаются.

Нетривиальное сэмплирование реализовано во фрагментном шейдере сферы. Исходные UV масштабируются, после чего синус и косинус задают

изменяющееся во времени волновое искажение. Формируются две пары координат: первая смещается в одном направлении, вторая дополнительно поворачивается и движется в другом. По этим координатам выполняются две выборки одного и того же изображения.

Цвета двух выборок смешиваются с коэффициентом, зависящим от времени и координаты v. Полученный цвет умножается на изменяющийся во времени вектор оттенка. Параметр силы эффекта задаёт смешивание обычной и обработанной выборок; при нулевой силе сохраняется обычное текстурирование. Итоговое альбедо затем участвует в расчёте освещения по модели Блинна–Фонга.

Управление эффектом выполняется через push-константы фрагментного шейдера. Структура размером 16 байт содержит флаг включения, время, силу эффекта и масштаб UV. Перед отрисовкой каждого объекта она передаётся командой vkCmdPushConstants: для сферы флаг определяется интерфейсом, для пола всегда равен нулю. Параметры сэмплирования не включаются в структуру свойств модели в uniform-буфере.

Время эффекта накапливается по формуле $t = t + \Delta t \cdot speed$ независимо от анимации движения сферы. В разделе Sphere sampling доступны переключатель Animated sampling и параметры Effect speed, Effect strength и UV scale. Нулевая скорость останавливает изменение текстуры. В разделах Sphere material и Floor material выпадающие списки Texture независимо выбирают материал каждого объекта, а Texture tint задаёт его оттенок.

В каждом кадре обновляются параметры сцены и обе структуры моделей. После начала прохода рендеринга привязываются основной графический конвейер и набор сцены. Для каждого объекта выбираются набор текстуры и динамическое смещение, передаются push-константы, привязываются вершинный и индексный буферы и вызывается vkCmdDrawIndexed. Изменения интерфейса отражаются на изображении в реальном времени.

Результаты

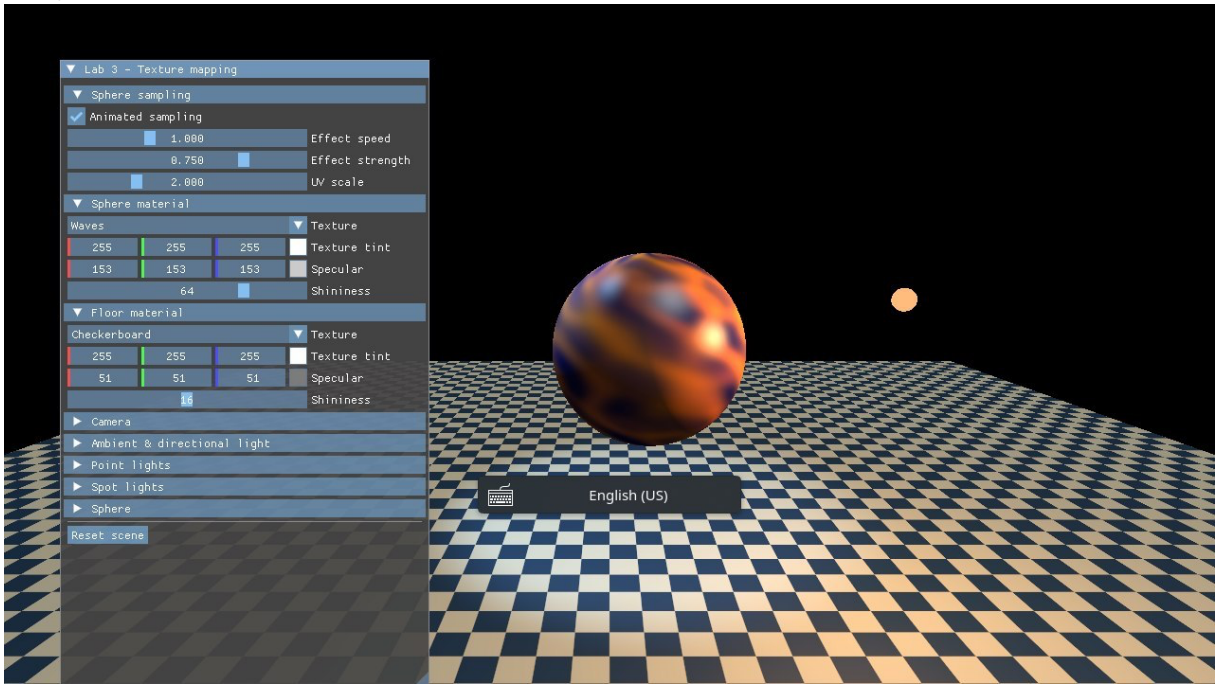


Рис. 1. Начальное состояние сферы.

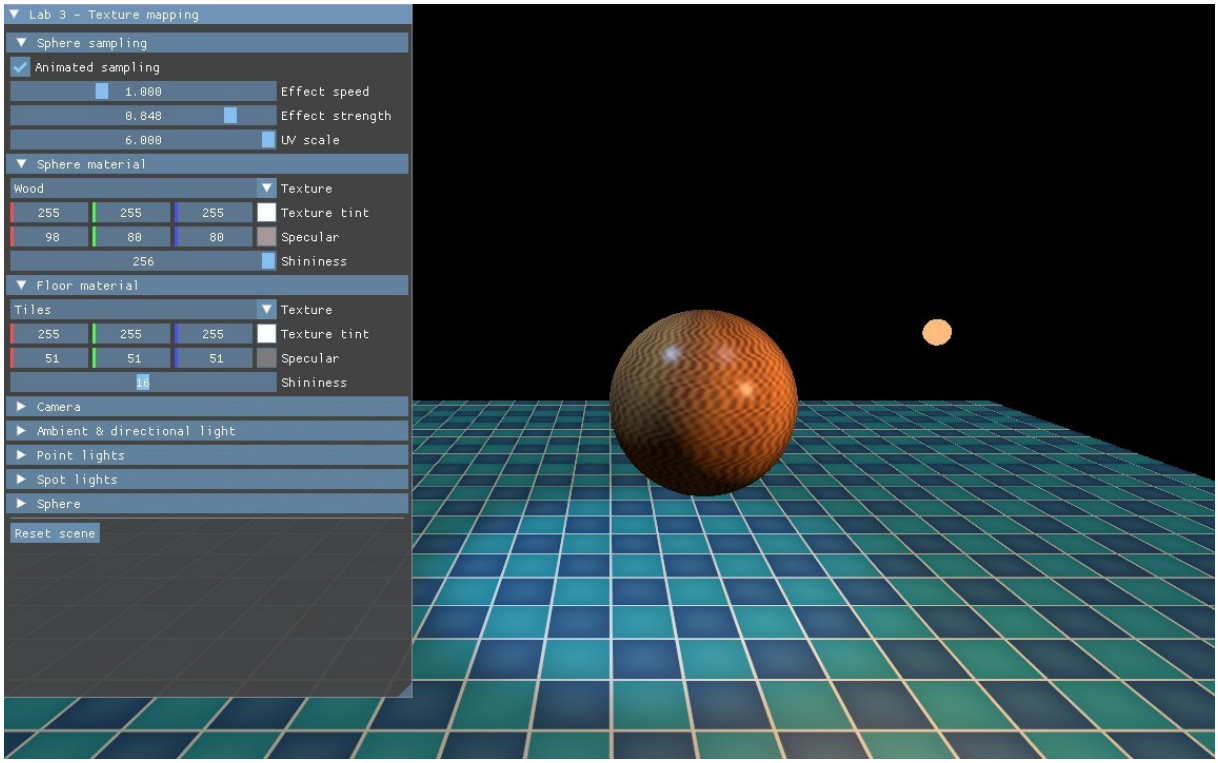


Рис. 2. Измененные параметры.

Выводы

В ходе лабораторной работы реализовано текстурирование сферы и пола изображениями, загружаемыми из файловой системы с помощью libpng. Освоены формирование UV-координат, создание изображений и сэмплера Vulkan, перенос текстур и геометрии в память GPU через staging-буферы. Реализованы отдельные наборы дескрипторов для шести текстур и общий динамический uniform-буфер свойств моделей с выбором структуры посредством смещения. Для сферы разработан анимированный эффект, объединяющий искажение UV, несколько выборок одного изображения и модуляцию цвета. Включение и параметры эффекта передаются через push-константы. Интерфейс ImGui позволяет независимо выбирать текстуры объектов и изменять параметры сэмплирования в реальном времени. Выполнены базовые требования и первые два дополнительных задания.